Sunbeam Institute of Information Technology
Pune and Karad

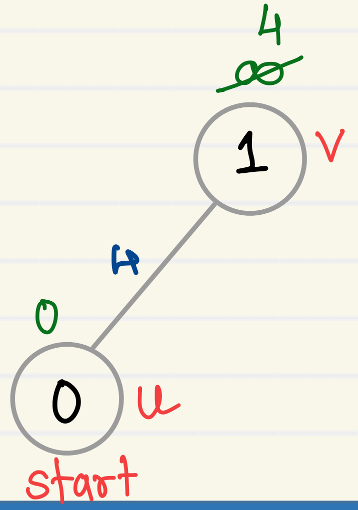
Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- **Initialisations**
  - Create a set mst to keep track of vertices included in MST.
  - Also keep track of parent of each vertex. Initialise parent of each vertex to -1.
  - Assign a key to all vertices in the input graph. Key of all vertices should be initialised to INF. The start vertex key should be 0.

```
boolean mst[vertexCount]
int parent[vertexCount]
int key[vertexCount]
```

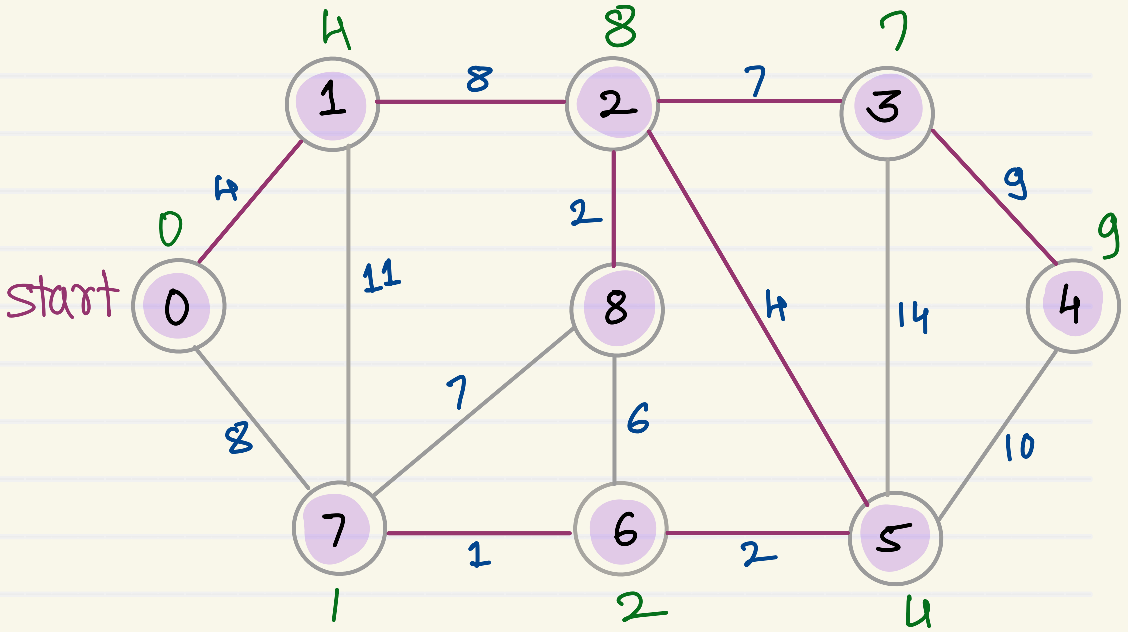


- **Algorithm**

  While mst doesn’t include all vertices
  1. Pick a vertex u which is not there in mst and has minimum key.
  2. Include vertex u in mst.
  3. Update key and parent of all adjacent vertices of u which are not there in mst
     i. For each adjacent vertex v, if weight of edge u-v is less than the current key of v, then update the key as weight of u-v.
     ii. Record u as parent of v.

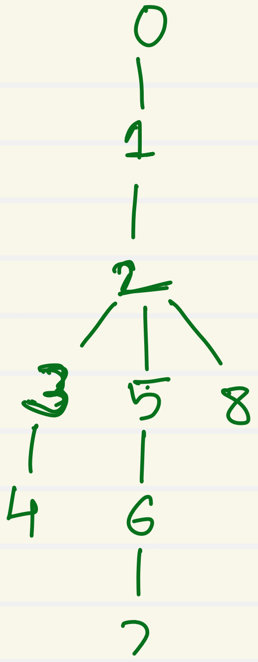
```
if(adjMatrix[u][v] < key[v]) {
    key[v] = adjMatrix[u][v];
    parent[v] = u;
}
```

Prim's MST



mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	9	3
5	4	2
6	2	5
7	1	6
8	2	2

Wt = 37



mst	Key	Parent
0	0	-1
1	4	0
2	∞	-1
3	∞	-1
4	∞	-1
5	∞	-1
6	∞	-1
7	8	0
8	∞	-1

mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	∞	-1
4	∞	-1
5	∞	-1
6	∞	-1
7	8	0
8	∞	-1

mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	∞	-1
5	4	2
6	∞	-1
7	8	0
8	2	2

mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	∞	-1
5	4	2
6	6	8
7	7	8
8	2	2

mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	10	5
5	4	2
6	2	5
7	7	8
8	2	2

mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	10	5
5	4	2
6	2	5
7	1	6
8	2	2

mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	10	5
5	4	2
6	2	5
7	1	6
8	2	2

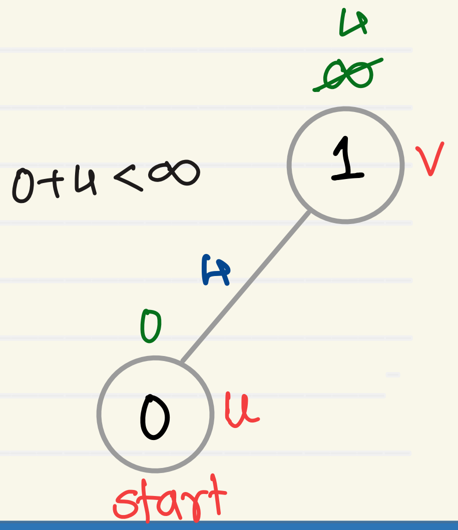
mst	Key	Parent
0	0	-1
1	4	0
2	8	1
3	7	2
4	9	3
5	4	2
6	2	5
7	1	6
8	2	2

Dijkstra's Algorithm

(single Source shortest path/distance)

- **Initialisations**
  - Create a set spt to keep track of vertices included in shortest path tree.
  - Track distance of all vertices in the input graph. Distance of all vertices should be initialised to

```
boolean spt[vertexCount]
int dist[vertexCount]
```



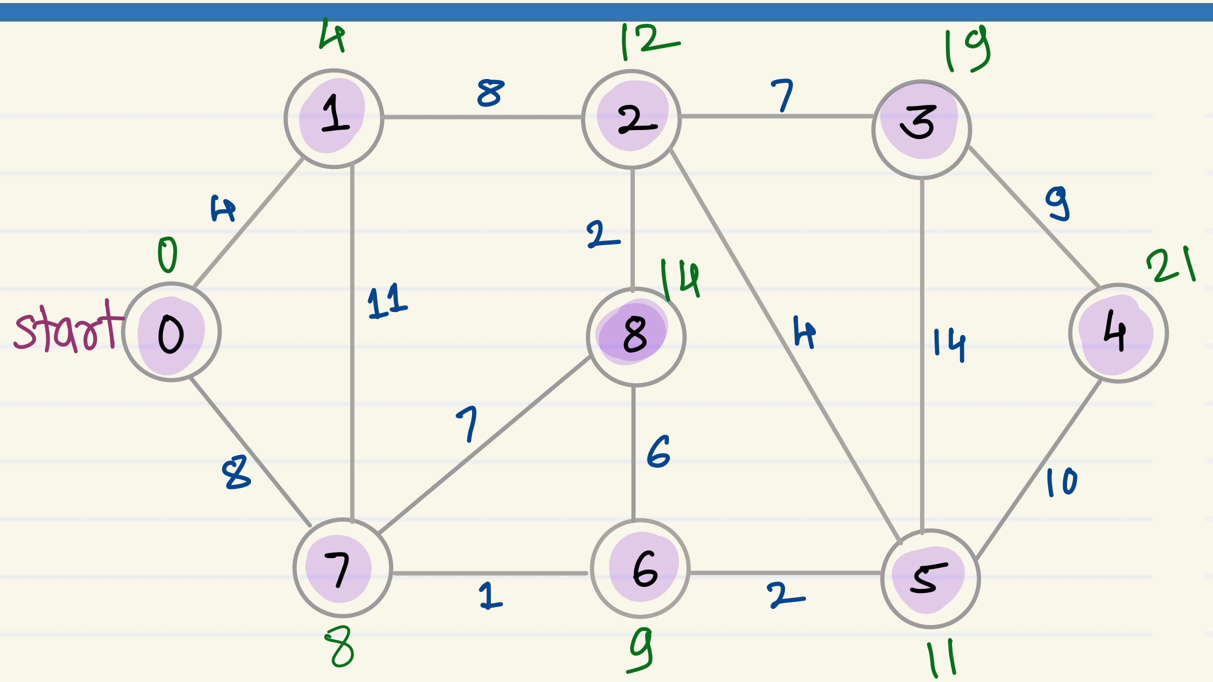
- **Algorithm**

  While spt doesn't include all vertices
  1. Pick a vertex u which is not there in spt and has minimum distance.
  2. Include vertex u in spt.
  3. Update distances of all adjacent vertices of u which are not there in spt

     For each adjacent vertex v, if distance of u + weight of edge u-v is less than the current distance of v, then update it's distance as distance of u + weight of edge u-v.

```
if(dist[u] + adjMatrix[u][v] < dist[v]){
    dist[v] = dist[u] + adjMatrix[u][v];
}
```

Dijkstra's Algorithm



spt	Dist
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14

spt	Dist
0	0
1	4
2	∞
3	∞
4	∞
5	∞
6	∞
7	8
8	∞

spt	Dist
0	0
1	4
2	12
3	∞
4	∞
5	∞
6	∞
7	8
8	∞

spt	Dist
0	0
1	4
2	12
3	∞
4	∞
5	∞
6	9
7	8
8	15

spt	Dist
0	0
1	4
2	12
3	∞
4	∞
5	11
6	9
7	8
8	15

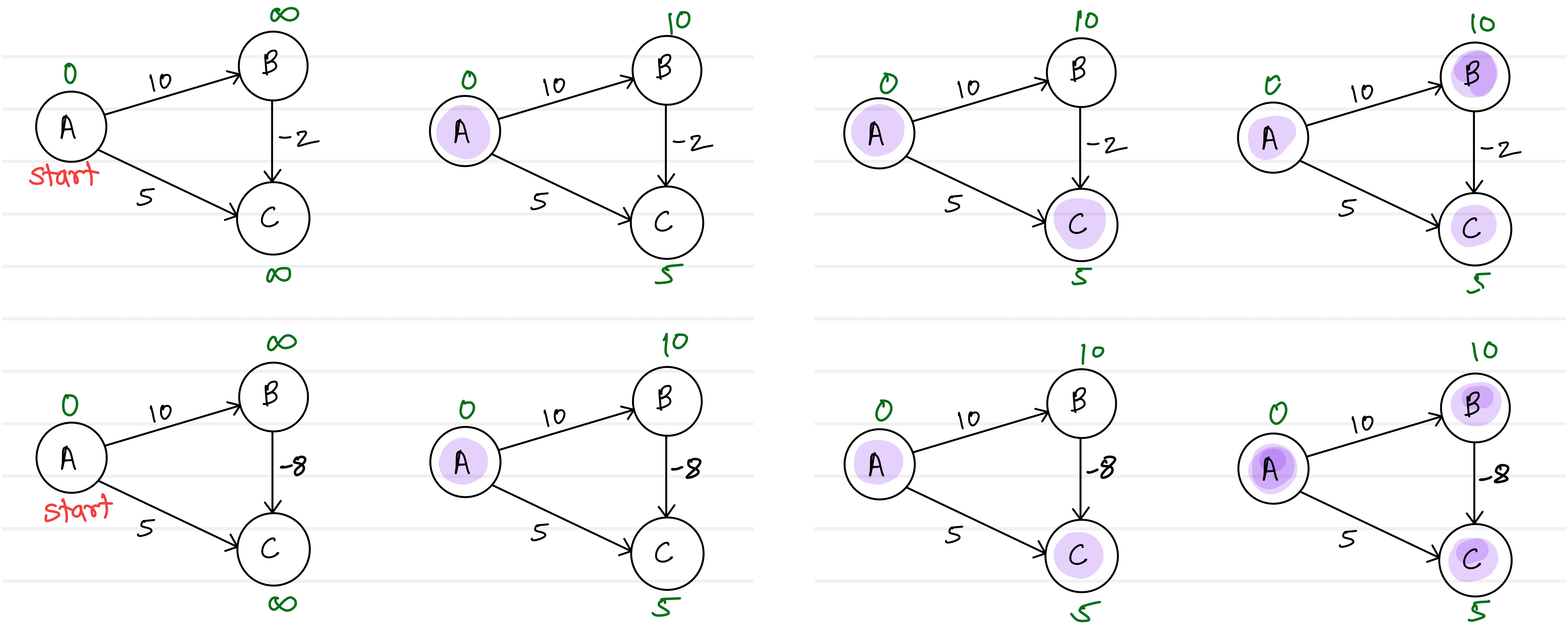
spt	Dist
0	0
1	4
2	12
3	25
4	21
5	11
6	9
7	8
8	15

spt	Dist
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14

spt	Dist
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14

spt	Dist
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14

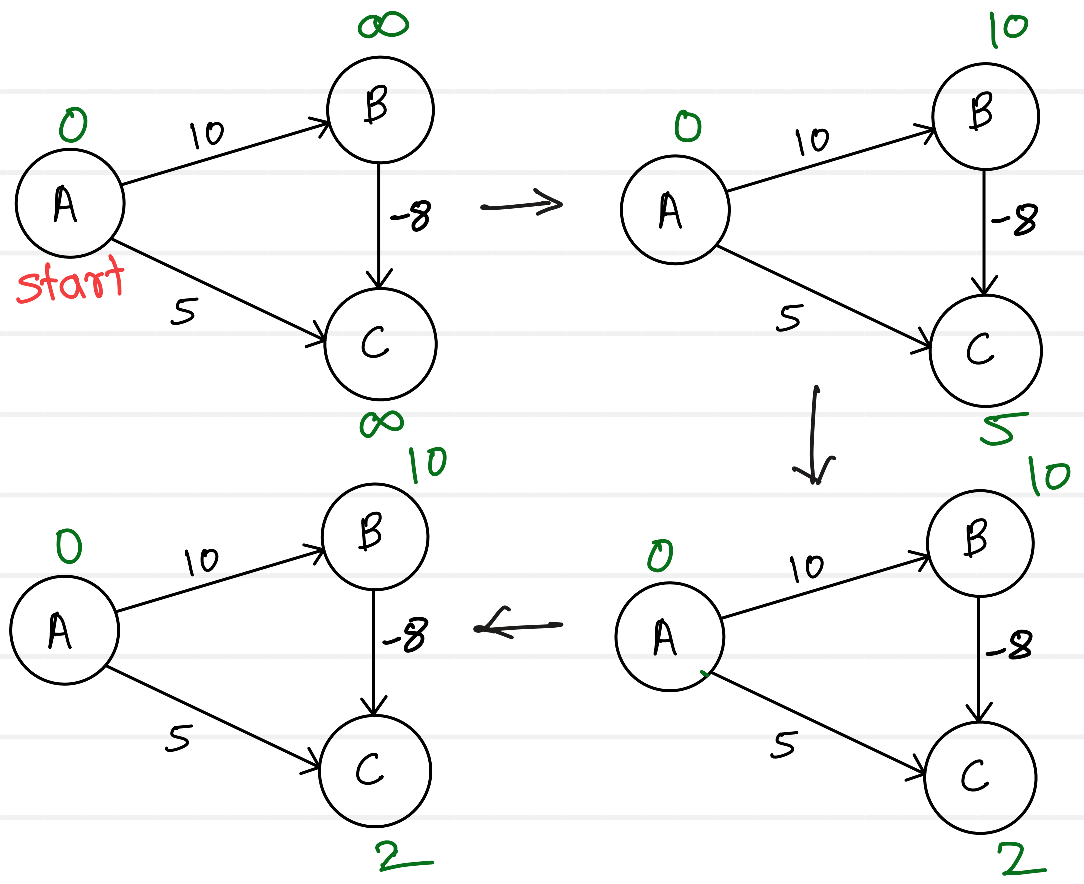
Dijkstra’s Algorithm



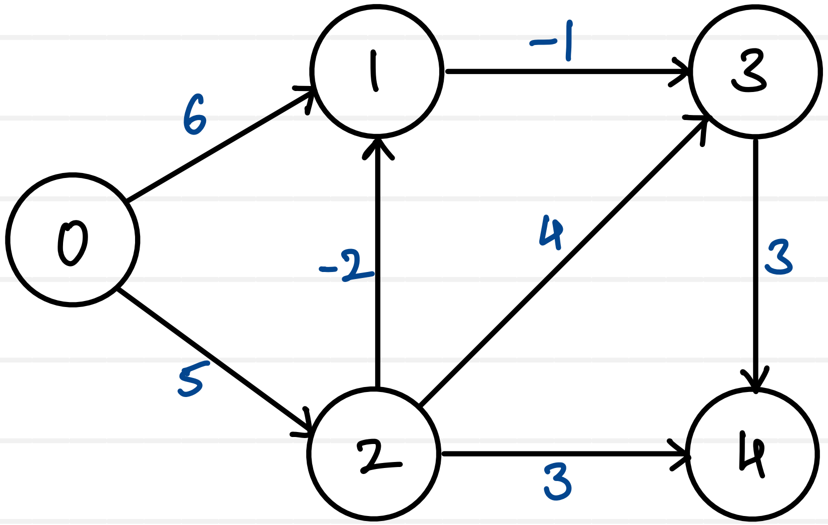
Dijkstra's algorithm may get fail, if graph has -ve edges

Bellman Ford Algorithm

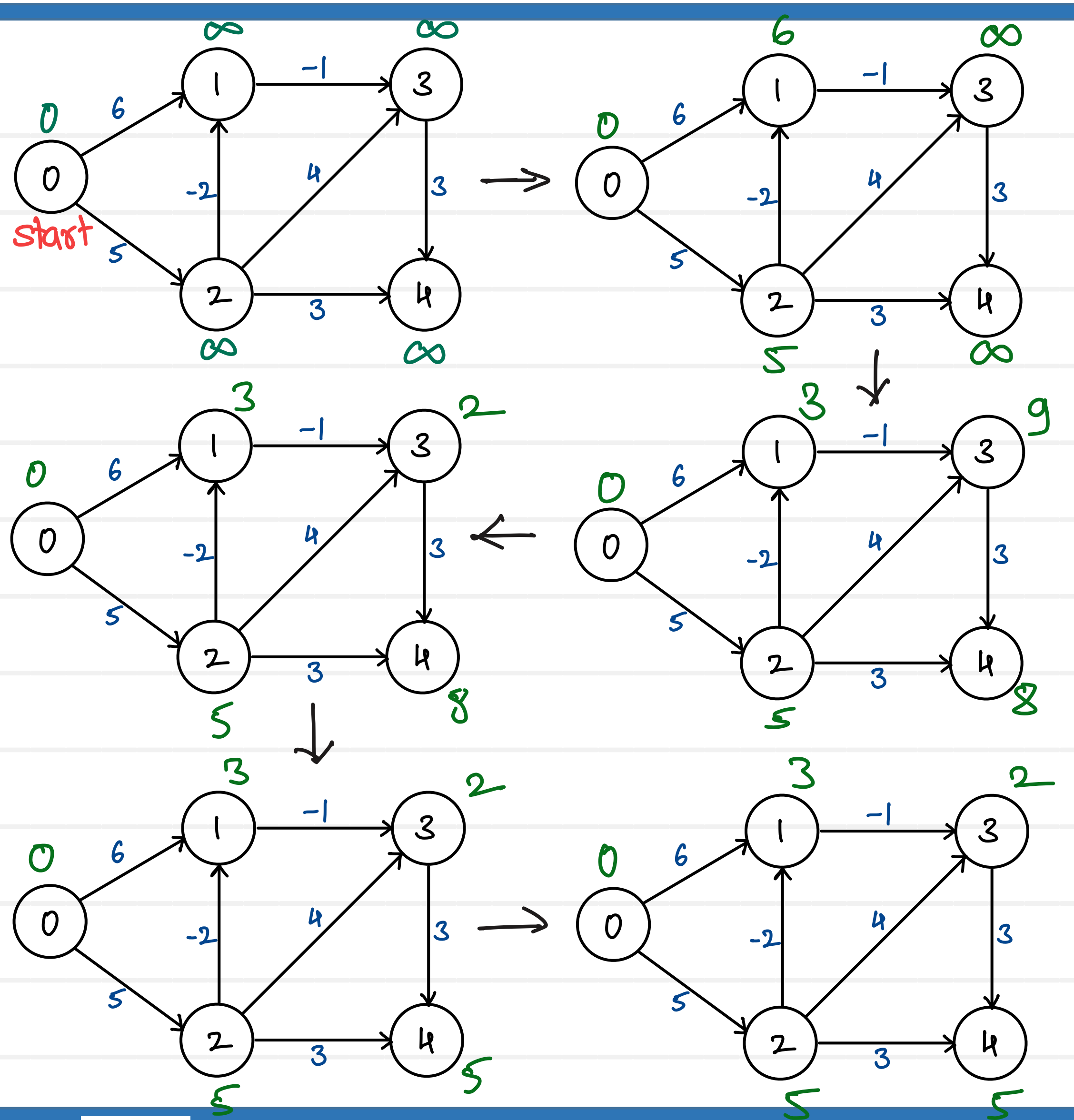
1. Initializes distances from the source to all vertices as infinite and distance to the source itself as 0.
2. Calculates shortest distance V-1 times:
   For each edge u-v,
   if dist[v] > dist[u] + weight of edge u-v,
   then update dist[v], so that
   dist[v] = dist[u] + weight of edge u-v.
3. Check if negative edge cycle in the graph:
   For each edge u-v,
   if dist[v] > dist[u] + weight of edge (u,v),
   then graph has -ve weight cycle.

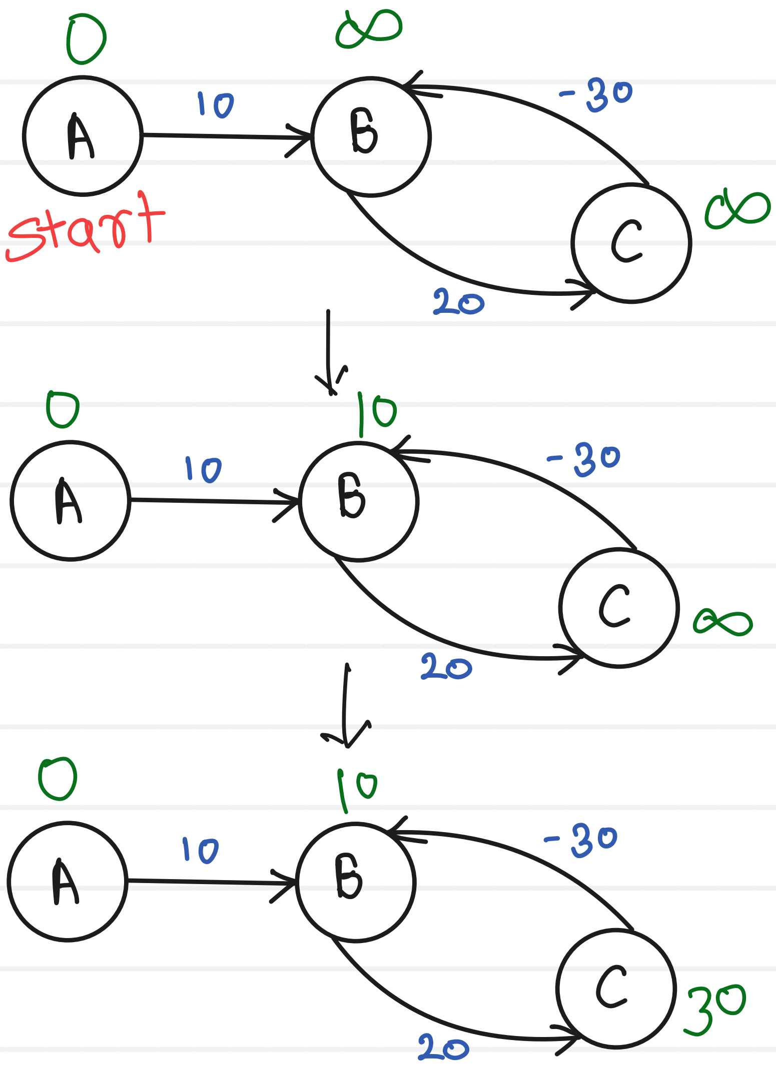


Bellman Ford Algorithm

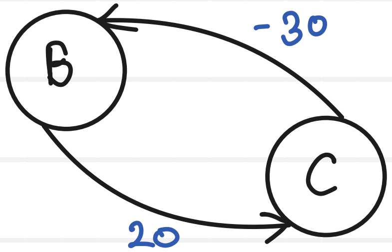


	0	1	2	3	4
	0	∞	∞	∞	∞
Pass 1	0	6	5	∞	∞
Pass 2	0	3	5	9	8
Pass 3	0	3	5	2	8
Pass 4	0	3	5	2	5





- Bellman ford will get fail when graph has -ve edge cycle



cycle wt = 20 - 30
= -10

(cycle with -ve weight is -ve edge cycle)